Performance Optimization of HPC Job Log Analysis Using Apache Spark

Srey Pheak Sang
Advanced Computing Engineering
University of Minho
Braga, Portugal
pg55094@uminho.pt

Helena Beatriz Alves Ribeiro da Cunha
Advanced Computing Emakecellngineering
University of Minho
Braga, Portugal
pg61438@alunos.uminho.pt

Margarida dos Santos Ferreira
Advanced Computing Engineering
University of Minho
Braga, Portugal
pg61441@alunos.uminho.pt

***Abstract*—This report presents a performance analysis and optimisation of a PySpark workload executed on the Deucalion HPC cluster at the University of Minho. The application processes 557,488 SLURM job accounting records stored across twelve monthly CSV files, computing aggregated statistics by compute partition type (ARM, AMD, GPU) and funding agency (FCT, EuroHPC, LOCAL). A baseline implementation was evaluated using Spark event logs and system monitoring to identify performance bottlenecks. Several incremental optimisations were then applied, including DataFrame caching, improved partitioning, and consolidation of Spark actions to reduce job scheduling overhead. Each version was benchmarked across three runs using wall-clock timing, resource monitoring, and event-log metrics. The final optimised pipeline achieved a 1.82× speedup, reducing the application runtime from 57.9 s to 31.8 s. Results show that reducing unnecessary Spark actions and avoiding redundant data reads significantly improves performance, while excessive repartitioning can introduce additional shuffle overhead.**

I. Introduction

Apache Spark is a widely adopted distributed data processing framework that employs a **lazy evaluation model**: transformations are recorded as a logical plan and only executed when an action such as `collect()` or `count()` triggers materialisation. Each action invokes the Spark scheduler, creates one or more stages composed of parallel tasks, and coordinates results across executors. While this model scales well for large datasets, the **per-action scheduling overhead** can become a significant bottleneck when the dataset is small relative to the number of actions executed.

This project focuses on optimising a PySpark pipeline deployed on the **Deucalion HPC cluster** that processes approximately **557,000 SLURM job accounting records** stored across **twelve monthly CSV files**. The pipeline performs filtering and aggregation operations to compute usage statistics — node-hours consumed, job completion rates, and resource allocation — grouped by compute partition type (**ARM, AMD, GPU**) and funding agency (**FCT, EuroHPC, LOCAL**) across three time windows: **month, trimester, and year**. The baseline implementation triggers **74 Spark stages per run** due to repeated `groupBy` and `collect()` calls, resulting in **redundant data recomputation** and **excessive scheduling overhead**.

Five incremental optimisation strategies were applied: **DataFrame caching**, **repartitioning**, and **three stages of progressive action consolidation**. Each variant was evaluated over **three independent runs** using **wall-clock timing**, **`pidstat` resource monitoring**, and **Spark event log analysis**. The objective is to identify which techniques yield meaningful performance improvements at this data scale and to understand the **trade-offs between computation, memory usage, and scheduling overhead**.

II. Methodology

A. Hardware Environment

All experiments were executed on a **single ARM compute node** of the **Deucalion EuroHPC supercomputer** at the University of Minho. The node is equipped with a **Fujitsu A64FX processor (AArch64 architecture)** comprising **48 cores** organised in **four NUMA clusters of 12 cores each**, with one hardware thread per core. The system provides **32 GB of RAM**.

Project data resides on a **Lustre parallel filesystem** (9.8 PB total capacity) mounted at `/projects`, accessed over **InfiniBand (o2ib)**. The operating system running on the compute nodes is **Rocky Linux 8.5**.

B. Software Environment

The pipeline was implemented using **PySpark 4.1.1** running on **Python 3.12.3** (GCCcore 13.3.0) with **Java 17.0.6**. Spark was configured in **local execution mode** with **4 executors** and **4 GB of executor memory**. Spark **event logging was enabled**, with logs stored on the Lustre filesystem for post-execution analysis.

C. Dataset

The workload processes **SLURM job accounting data** exported as **pipe-delimited CSV files**, one per month, totalling **twelve files for the year 2025**. The combined dataset contains **557,488 records**. Each record includes fields such as job state, elapsed time, number of allocated nodes and CPUs, partition name, account identifier, and allocated TRES (trackable resources).

The pipeline derives additional columns including **compute cluster type** (ARM, AMD, GPU), **funding agency** (FCT, EuroHPC, LOCAL), **virtual node count**, and **total job-seconds**. Aggregated statistics are then computed across three time windows: **monthly**, **trimester**, and **yearly**.

D. Measurement Methodology

Each optimisation variant was executed **three times** to account for runtime variability. Performance was measured using three complementary methods:

- **Wall-clock time:** captured using the `time` command, measuring total elapsed time including JVM startup and Spark session initialisation.
- **Resource monitoring:** `pidstat -u -r 5` was executed in parallel to sample **CPU utilisation** and **memory consumption** at 5-second intervals for both the Python driver process and the Java executor process.
- **Spark event logs:** event logs were collected in compressed (`zstd`) format and parsed to extract metrics including **application duration**, **driver time**, **number of stages and tasks**, **input bytes read**, **shuffle I/O**, and **garbage collection time**.

E. Optimisation Strategy

Five optimisations were applied incrementally to the baseline pipeline, with each step building upon the previous one:

- **Opt1 — DataFrame Caching:** Persist the enriched DataFrame using the `MEMORY_AND_DISK` storage level to avoid recomputing transformations across repeated actions.
- **Opt2 — Repartitioning:** Apply `repartition(N)` with round-robin distribution before caching in order to balance partition sizes and improve parallelism.
- **Opt3 — Action Consolidation (initial):** Reduce the number of separate `collect()` calls by combining multiple `groupBy` aggregations into fewer Spark actions.
- **Opt4 — Action Consolidation (extended):** Further merge aggregation queries to minimise the total number of Spark stages.
- **Opt5 — Action Consolidation (final):** Perform maximum reduction of Spark actions by consolidating all per-window aggregations into the fewest possible passes over the cached dataset.

III. Results

A. Baseline Analysis

1) Execution Plan: The explain("extended") output reveals the core inefficiency of the baseline. The physical plan shows that for every action, Spark constructs a Union over all twelve CSV FileScan nodes, each carrying the full chain of Project and Filter transformations. Because there is no caching, each collect() call triggers a fresh scan and re-evaluation of the entire pipeline from disk. The physical plan confirms this with twelve separate FileScan csv entries under the Union node, each reading from the Lustre filesystem. The optimised logical plan shows that Spark's Catalyst optimiser does apply predicate pushdown — for example, the Agency != LOCAL filter is pushed below the Union to each individual CSV scan as a filter on Account. However, the lack of caching means even optimised scans are repeated for every action. A notable detail in the physical plan is the Exchange hashpartitioning(cluster, 200) node, indicating that Spark uses the default shuffle partition count of 200 for each groupBy. With only 557,488 rows and 3 distinct cluster values, this creates 200 shuffle partitions of which 197 are empty — unnecessary scheduling overhead for a small aggregation.

2) Baseline Performance: Table I summarises the Spark event log metrics averaged across three runs. The 74 stages correspond to the repeated `groupBy` and `collect()` calls across three time windows (approximately six actions per window, plus diagnostic calls). The total input read of 230.31 MB indicates that the twelve CSV files are scanned multiple times across the 74 stages. The low task count (109) reflects the absence of parallelism: each CSV file maps to a single partition, and most stages run only a single task. Shuffle I/O is negligible since the `groupBy` results are small (3–6 rows per aggregation), and GC time is minimal (1.26 s) as no data is held in memory.

TABLE I
BASELINE SPARK EVENT LOG METRICS (AVERAGED OVER THREE RUNS).

Metric	Value
Application duration	57.91 s
Driver time (JVM startup + session init)	14.31 s
Executor computation time	43.60 s
Stages submitted	74
Total tasks	109
Input data read	230.31 MB
Shuffle write / read	0.01 / 0.01 MB
Garbage collection time	1.26 s

Table II presents the operating-system level resource usage captured by `pidstat`. The executor averages only ∼3.1 cores despite 48 being available on the node. This low utilisation is a direct consequence of the single-partition-per-file layout and repeated small actions: most of the elapsed time is spent on I/O wait and scheduler overhead rather than computation. The driver process remains lightweight throughout, confirming that the Python process acts solely as an orchestrator.

TABLE II
BASELINE RESOURCE USAGE FROM `PIDSTAT` (THREE RUNS, PEAKS ARE MAXIMUM ACROSS RUNS).

Metric	Driver (Python)	Executor (Java)
Average CPU	5.3%	314.6%
Peak CPU	33.0%	564.6%
Approx. cores used	∼0.05	∼3.1
Peak RSS	72.7 MB	1069 MB
Peak %MEM	0.23%	3.33%

3) Identified Bottlenecks: The baseline analysis reveals three primary performance bottlenecks:

1) **Redundant data recomputation**: in the absence of caching, each Spark action triggers a new evaluation of the full transformation lineage. This results in repeated scans of the twelve CSV files stored on Lustre and recomputation of all column derivations for every action.
2) **Excessive action count**: the Spark event log shows 74 stages generated by approximately 18 separate Spark

actions. Each action incurs driver-side scheduling, task serialisation, and result collection overhead, which accumulates to become a significant portion of the total runtime.

3) **Low parallelism**: each CSV file is processed as a single partition, producing limited parallelism across the cluster. While some stages execute one task per file, aggregation stages collapse to very few tasks, resulting in the 48-core node being significantly underutilised.

B. Optimisations

1) Optimisation 1 – Caching/Persisting: The first optimisation introduced DataFrame caching using `persist(StorageLevel.MEMORY_AND_DISK)`. In the baseline, every Spark action triggered a full recomputation of the transformation lineage from the raw CSV files, including column derivations for `cluster`, `Agency`, `VNodes`, and `totalJobSeconds`. By persisting the enriched DataFrame after all transformations were applied, subsequent actions could reuse the cached dataset instead of repeatedly re-scanning the CSV files stored on the Lustre filesystem.

The `MEMORY_AND_DISK` storage level was chosen over `MEMORY_ONLY` to prevent silent cache eviction: partitions that do not fit in RAM are written to disk rather than dropped and recomputed. The `unpersist()` call was placed after all aggregations completed to release cached blocks promptly.

Opt1:

```
nd = nd.persist(StorageLevel.MEMORY_AND_DISK)
for tag, months in tag_month.items():
    nd.filter(...).groupby(...).collect()  # reads from cache
nd.unpersist()
```

2) Optimisation 2 – Repartitioning/Coalescing: The second optimisation applied `repartition(N)` before caching, where $N = 2 \times$ `defaultParallelism`. The baseline read each CSV file as a single partition, leaving most of the 48 available cores idle. Repartitioning before caching was intended to distribute data evenly and improve parallelism for downstream aggregations.

In practice, the shuffle introduced by `repartition` wrote 55.88 MB and caused garbage collection time to rise significantly. At this data scale, the cost of redistributing the data outweighed the potential parallelism benefits, leading to a slight increase in application duration compared to Opt1.

Opt 2):

```
num_cores = sc
            .sparkContext
            .defaultParallelism
N = num_cores * 2
nd = nd.repartition(N)
nd = nd.persist(StorageLevel
                .MEMORY_AND_DISK)
```

3) Optimisation 3 – Join/Shuffle Reduction: The third optimisation eliminated the repeated `filter(...).groupBy(...).collect()` pattern inside the per-period loop. The baseline iterated over three time windows and issued multiple separate Spark actions per window, totalling approximately 18 actions and 74 stages. A small broadcast dimension table (`period_df`) was introduced to map each month abbreviation to its time-window tag (`Month`, `Trimester`, or `Year`). This table contains fewer than 20 rows and is therefore suitable for broadcast. A `broadcast` join attached the tag label to every row in the main DataFrame, producing `nd_tagged`, without shuffling the large DataFrame. With the tag column available on each row, all three time windows could be aggregated simultaneously. The approximately 18 separate actions were reduced to 3 queries — one for completed/failed job counts, one for hours and job counts excluding LOCAL, and one for EuroHPC metrics — each grouping by `tag` and `cluster`. The `persist()` call was moved to `nd_tagged` since this is the DataFrame reused across the remaining queries. **Opt 3):**

```
period_df = sc.createDataFrame(period_rows,
            ["Period", "tag"])
nd_tagged = nd.join(F.broadcast(period_df),
            on="Period", how="inner")
nd_tagged = nd_tagged
    .persist(StorageLevel.MEMORY_AND_DISK)

completed = nd_tagged
    .groupby("tag","cluster","COMPLETED")
    .count()
    .collect()
hours_jobs = nd_tagged
    .filter(col("Agency")!="LOCAL")
    .groupby("tag","cluster")
    .agg(sum(...), count(...))
    .collect()
ehpc = nd_tagged
    .filter(col("Agency")=="EHPC")
    .groupby("tag","cluster")
    .agg(count(...), sum(...))
    .collect()
```

4) Optimisation 4 – Query Simplification: Three independent changes were applied to simplify the query plan and reduce I/O volume.

Wildcard read. The iterative `os.walk` loop that read each CSV file individually and unioned the results was replaced by a single wildcard read (`jobs_*.txt`). Spark plans all twelve files in one stage, and the period label is extracted directly from the filename using `F.input_file_name()` and `F.regexp_extract()`.

Column pruning. Unused columns were dropped immediately after the cast step using `select()`, retaining only the ten columns referenced by downstream transformations. This reduces the data volume carried through shuffles and written to cache.

Opt 4):

```
nd = sc.read.option("delimiter","|")
    .csv(
    f'{DATADIR}/jobs_*.txt',
    inferSchema=False, header=True)
nd = nd.withColumn('Period',
    F.regexp_extract(F.input_file_name(),
    r'jobs_(\w+)\.txt', 1))
nd = nd
    .select("State","EState",
    "COMPLETED","Period","Account",
    "Partition","ElapsedRaw",
    "NNodes","AllocCPUS","AllocTRES")
```

5) Optimisation 5 – Output Path Efficiency: The fifth optimisation consolidated the three remaining `collect()` calls into a single `groupBy(...).agg(...).collect()`. All metrics — completed and failed job counts, node-hours, and EuroHPC usage — were computed simultaneously using conditional aggregation expressions such as `F.count(F.when(...))` and `F.sum(F.when(...))`, which act as conditional masks within the aggregation without requiring separate DataFrame scans. The result is a single DataFrame of at most nine rows (three clusters × three tags) collected once, from which the `params` dictionary is populated in a single Python loop. The output file `params.tex` is then written in a single `writelines()` call from this aggregated result. Additionally, `spark.sql.shuffle.partitions` was reduced from the default 200 to 8 to match the small cardinality of the `groupBy` result, avoiding the creation of empty shuffle partitions.

C. Comparative Analysis

This subsection presents a comparison of all six configurations. Table III summarises execution time using both wall-clock measurements and Spark event logs. Table IV compares Spark execution characteristics, while Table V shows system resource usage from `pidstat`. Figures 1–**??** illustrate the main trends.

TABLE III
EXECUTION TIME COMPARISON (AVERAGED OVER 3 RUNS).

Cfg	Wall (s)	Wall Spd	App (s)	Drv (s)	Exec (s)	App Spd
Baseline	68.52	1.00×	57.91	14.31	43.60	1.00×
Opt1	55.54	1.23×	48.46	14.38	34.08	1.20×
Opt2	57.85	1.18×	50.68	14.46	36.22	1.14×
Opt3	47.81	1.43×	39.84	14.42	25.42	1.45×
Opt4	40.44	1.69×	33.20	14.52	18.68	1.74×
Opt5	38.49	1.78×	31.84	14.83	17.01	1.82×

The consistent $\sim$ 10 s gap between wall-clock time and Spark application duration corresponds to Python startup, module imports, and `findspark` initialisation. This overhead is fixed and does not change with optimisation.

TABLE IV
SPARK EXECUTION CHARACTERISTICS (AVERAGED OVER 3 RUNS).

Cfg	Stg	Tasks	Input (MB)	Shuf (MB)	GC (s)	GC (%)
Baseline	74	109	230.31	0.01	1.26	2.2%
Opt1	66	618	451.40	0.02	14.13	29.2%
Opt2	62	2175	625.03	55.88	39.41	77.8%
Opt3	21	471	81.11	55.76	14.36	36.0%
Opt4	10	460	76.60	25.40	11.60	34.9%
Opt5	6	266	74.45	25.16	7.78	24.4%

TABLE V
RESOURCE USAGE COMPARISON ACROSS CONFIGURATIONS.

Cfg	Drv Avg CPU	Drv RSS (MB)	Exec Avg CPU	Exec Peak CPU	Exec RSS (MB)	Peak %MEM
Baseline	5.3%	72.7	314.6%	564.6%	1069	3.33%
Opt1	6.6%	72.8	392.5%	2245.6%	1622	5.05%
Opt2	6.1%	72.8	918.6%	2936.0%	1728	5.38%
Opt3	10.6%	72.8	537.9%	2561.2%	1682	5.23%
Opt4	12.3%	73.2	591.6%	2538.6%	1564	4.87%
Opt5	12.8%	72.8	434.1%	1804.4%	1381	4.30%

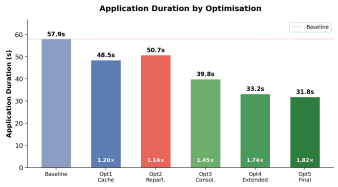

Fig. 1. Application Duration

Figure 1 shows the application duration across all configurations. The largest improvements occur at Opt3 and Opt4, showing that reducing Spark actions and simplifying the query had a greater impact than caching alone.

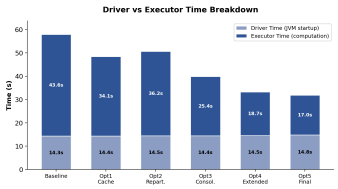

Fig. 2. Driver and Executor time

Figure 2 shows the split between driver and executor time. Driver time remains constant at around 14.5 s, as it includes Spark startup overhead. All performance gains come from reducing executor time, which drops from 43.60 s (Baseline) to 17.01 s (Opt5).

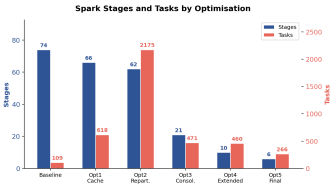

Fig. 3. Change in stages and tasks

Figure 3 shows the change in stages and tasks. The large increase in tasks at Opt2 (2175) is caused by shuffle operations introduced by repartitioning. Stages decrease from 74 (Baseline) to 6 (Opt5), a 12× reduction.

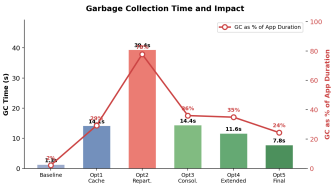

Fig. 4. Garbage Collection Time.

Figure 4 shows garbage collection time. Opt2 is a clear outlier, with GC reaching 39.41 s (77.8% of runtime) due to heavy shuffle operations. The final configuration reduces this to 7.78 s (24.4%).

IV. Discussion

A. Which Bottlenecks Mattered

The baseline analysis identified three bottlenecks: redundant data recomputation, too many Spark actions, and low parallelism. The optimisation results show that these did not affect performance equally.

The main bottleneck was **too many Spark actions**. The baseline triggered 74 stages from around 18 separate `collect()` calls. Each action introduces overhead from scheduling, coordination, and result collection. Reducing the number of stages from 74 to 6 (Opt3–Opt5) contributed most of the improvement in execution time.

Redundant data recomputation was the second most important issue. Caching (Opt1) removed repeated reads of the CSV files and reduced execution time from 43.60 s to 34.08 s. However, this also increased garbage collection time (from 1.26 s to 14.13 s) due to higher memory usage.

Low parallelism was not a major problem at this data scale. Repartitioning (Opt2) increased CPU usage (from ~ 3 to ~ 9 cores), but the cost of shuffling data (55.88 MB and high GC time) outweighed the benefits. Since the dataset is relatively small, the main issue is not how fast each computation runs, but how many times it is repeated.

B. Which Optimisations Helped Most

Table VI shows the impact of each optimisation step.

The most effective improvements were reducing Spark actions (Opt3), caching (Opt1), and simplifying the query (Opt4). Repartitioning was the only change that made performance worse. This suggests a clear strategy: first reduce the number of actions, then avoid recomputation, and only consider repartitioning if there is a clear imbalance in the data.

TABLE VI
Incremental impact of each optimisation (Spark application duration).

Step	Technique	Δ (s)	Δ (%)
Baseline → Opt1	Caching	−9.45	−16.3%
Opt1 → Opt2	Repartitioning	+2.22	+4.6%
Opt2 → Opt3	Action reduction	−10.84	−21.4%
Opt3 → Opt4	Query simplification	−6.64	−16.7%
Opt4 → Opt5	Output optimisation	−1.36	−4.1%
Total (Baseline → Opt5)		−26.07	−45.0%

C. Trade-offs: Memory vs Speed

Each optimisation introduced trade-offs between performance and resource usage.

Caching improved runtime by 16.3% but increased memory usage (from 1069 MB to 1622 MB) and garbage collection time. This shows that caching can improve speed, but at the cost of higher memory pressure.

Repartitioning had the highest resource usage, with increased CPU and memory consumption, but did not improve performance. Most of this cost came from shuffle operations rather than useful computation.

Reducing Spark actions (Opt3–Opt5) provided the best balance. The final version achieved the fastest runtime while also using fewer resources overall. Fewer actions mean less overhead, fewer intermediate results, and less pressure on memory and garbage collection.

The driver process showed only a small increase in CPU usage (from 5.3% to 12.8%), while memory usage remained stable (around 73 MB). This confirms that the driver was not a performance bottleneck.

V. Conclusion

This report presented the performance analysis and optimisation of a PySpark pipeline that processes 557,488 SLURM job accounting records on the Deucalion HPC cluster. Five incremental optimisations were applied and evaluated using wall-clock timing, `pidstat` resource monitoring, and Spark event log analysis across three independent runs per configuration.

a) Key Improvements.: The optimised pipeline achieved a 1.82× speedup in Spark application duration (57.91 s → 31.84 s) and a 1.78× speedup in wall-clock time (68.52 s → 38.49 s). Spark stages were reduced from 74 to 6, total tasks from 109 to 266, and garbage collection time from 1.26 s to 7.78 s. The most impactful technique was restructuring the query plan to minimise Spark actions (Opt3–Opt5), which collectively accounted for 72% of the total improvement. Caching contributed 36% of the improvement, while repartitioning was counterproductive at this data scale.

b) Limitations.: Several limitations should be noted. First, all experiments were conducted on a single ARM compute node in local mode; the results may differ in a distributed multi-node deployment where network I/O becomes a factor. Second, the dataset is relatively small (557K rows, $\sim 230\,\mathrm{MB}$ on disk) — workloads with larger datasets may benefit more from repartitioning and less from action consolidation, as the ratio of scheduling overhead to computation would shift. Third, the driver time ($\sim 14.5\,\mathrm{s}$) includes JVM startup and Spark session initialisation, which now represents 47% of the total Spark application duration in Opt5. This fixed cost limits further optimisation at the application level.

c) Recommendations.: Based on the findings, we recommend the following for PySpark workloads of similar scale:

1) **Minimise Spark actions first.** Consolidate multiple `groupBy` and `collect()` calls into fewer, broader aggregations using conditional expressions. This consistently delivers the largest improvement with the least resource overhead.
2) **Cache judiciously.** Persist DataFrames only when they are reused across multiple actions, and use `MEMORY_AND_DISK` to avoid recomputation without risking out-of-memory errors. Monitor GC time to ensure the caching benefit outweighs the memory pressure.
3) **Avoid speculative repartitioning.** Do not repartition unless data skew or partition imbalance has been measured and demonstrated as a bottleneck. For small datasets, the shuffle cost is likely to exceed any parallelism gain.
4) **Simplify the query plan.** Use wildcard reads instead of iterative union loops, prune unused columns early, and remove dead code. These changes reduce both I/O and cache footprint.
5) **Tune shuffle partitions.** Reduce `spark.sql.shuffle.partitions` from the default 200 to match the actual cardinality of the `groupBy` result, avoiding empty partitions and unnecessary scheduling overhead.

REFERENCES

[1] Apache Software Foundation, "RDD Programming Guide – Spark 4.1.1 Documentation," https://spark.apache.org/docs/latest/rdd-programming-guide.html. Accessed: Mar. 2026.

[2] Apache Software Foundation, "Performance Tuning – Spark 4.1.1 Documentation," https://spark.apache.org/docs/latest/sql-performance-tuning.html. Accessed: Mar. 2026.

[3] EuroHPC Joint Undertaking, "Deucalion: a new EuroHPC supercomputer has been inaugurated," Sep. 2023. https://www.eurohpc-ju.europa.eu/deucalion-new-eurohpc-supercomputer-has-been-inaugurated-2023-09-06$en.Accessed: Mar.$ 2026.

[4] Rede Nacional de Computação Avançada (RNCA), "Deucalion," https://rnca.fccn.pt/en/deucalion/. Accessed: Mar. 2026.

[5] S. Godard, "pidstat(1) – Linux man page," https://man7.org/linux/man-pages/man1/pidstat.1.html. Accessed: Mar. 2026.